ProductHelper in the file product_helper.rb in the app/helpers directory. You don't have to remember all these details—the rails generate controller script creates a stub helper module automatically.

We can use helpers to clean up the application layout a bit. Currently we have the following:

```
<h3><%= @page_title || "Pragmatic Store" %></h3>
```

Let's move the code that works out the page title into a helper method. Because we're in the store controller, we edit the store_helper.rb file in app/helpers:

```
module StoreHelper
  def page_title
    @page_title || "Pragmatic Store"
  end
end
```

Now the view code simply calls the helper method:

```
<h3><%= page_title %></h3>
```

(We might want to eliminate even more duplication by moving the rendering of the entire title into a separate partial template, shared by all the controller's views, but we don't talk about partial templates until *Partial-Page Templates*.)

Helpers for Formatting and Linking

Rails comes with a bunch of built-in helper methods, available to all views. Here, we'll touch on the highlights, but you'll probably want to look at the Action View RDoc for the specifics—there's a lot of functionality in there.

Aside from the general convenience these helpers provide, many of them also handle internationalization and localization. In Chapter 15, *Task J: Internationalization*, we translated much of the application. Many of the helpers we used handled that for us, such as number_to_currency. It's always a good practice to use Rails helpers where they're appropriate, even if it seems just as easy to hard-code the output you want.